BENNETT UNIVERSITY
Greater Noida, Uttar Pradesh
Department of Computer Science & Engineering

RESEARCH PAPER
B.Tech · Computer Science & Engineering · 6th Semester

Optimizing API Performance Using AI-Based Predictive Request Management

Submitted by:	**Deepanshu Chauhan**
Enrollment No.:	E23CSEU1617
Programme:	B.Tech Computer Science & Engineering — 3rd Year
Submitted to:	Department of CSE, Bennett University
Date of Submission:	March 2026

Declaration

I am declaring that "Optimizing API Performance: AI-Predictive Request Management" Is an original work done by me and prepared for academic purposes. All sources have been referenced in the references provided. No part of this paper has been submitted in whole or in part elsewhere for any course or examination.

Deepanshu Chauhan
E23CSEU1617 · Bennett University · March 2026

ABSTRACT

APIs used by modern cloud applications are under pressure because of sudden increases in API request volume. These increases will affect the performance of the API by increasing response times, depleting available computer resources, and impacting the end-user experience. Reactive scaling methods are not effective when the speed of the increase in traffic outpaces the speed at which new capacity can be added to the infrastructure. This paper presents a confidence-calibrated hybrid predictive approach that combines an LSTM neural network and an XGBoost model to predict incoming API request volumes. Instead of sequentially or independently running these predictive methods, both models run in parallel and the output of each model will determine whether or not to scale an application's infrastructure based on its calibrated confidence score. The solution operates as seven sequential processing layers from the Apache Kafka ingestion system through the Apache Flink feature extraction system, the InfluxDB time-series database for storing feature data, the parallel execution of ML inference processing, and finally to the Redis-based action dispatch to the API. All actions are completed in real-time with an end-to-end latency less than 50 milliseconds. During the evaluation period of 95 million API requests spread over 45 consecutive days across 2 separate production systems, the confidence calibration approach reduced the 95th percentile latency from 247 milliseconds to 141 milliseconds (43% reduction), increased the cache hit ratio from 48.2% to 85.4% (37 percentage point increase), and reduced the average monthly infrastructure cost by 28%. All metrics evaluated during the study have a statistically significant difference at $p<0.001$. All components within the confidence calibrated predictive approach provided independent contributions to the results. In addition, a case study was performed on the Black Friday shopping event, and the predictive model was able to provide more than 8 minutes of lead time to scale the application during the 8.2x increase in traffic, which reduced the peak error rate from 12.3% to 1.1%.

Keywords: *API performance optimisation · predictive scaling · LSTM · XGBoost · ensemble learning · confidence calibration · real-time telemetry · cloud infrastructure*

KEY CONTRIBUTIONS

① Novel hybrid LSTM + XGBoost parallel architecture with confidence-calibrated ensemble gating for proactive API traffic management

② Seven-layer production pipeline (Kafka → Flink → InfluxDB → ML → Redis) validated at 95 million requests over 45 days

③ **43% p95 latency reduction · 37% cache hit improvement · 28% infrastructure cost savings — all at $p < 0.001$**

④ Ablation study per component + Black Friday real-world stress test with full reproducibility package

1. Introduction

The growing use of microservices by businesses has made API gateway performance a high priority engineering consideration. A mid-size SaaS platform may have more than 50K concurrent requests at any moment in time, each using latency expectations that its users would consider to be nonnegotiable. When traffic surges such as promotional campaigns, viral events or scheduled peaks during business hours occur faster than reactive auto-scaling can respond, service degradation will occur across dependent services with a compounding effect for every missed provisioning window.

When using auto-scaling policies that react to CPU / memory threshold breaches, they are traditionally based on lagging indicators. By the time that an auto-scaling event triggers due to a CPU threshold breach, the queuing depth has often breached the SLA for user-visible performance. The research community has recognised this gap for years, with relevant scholarly articles proposing methodologies for developing predictions of user request loads using ARIMA based predictions, gradient boosted trees, deep recurrent neural networks, etc. However, none of these methods have been successfully deployed in production, as production deployments of predictive API optimisation frameworks are really quite rare compared to the number of academic papers that have proposed them. Single model predictors have performance degradation issues when a distribution shift occurs, the predictions themselves do not have well defined uncertainty estimates and the architectures assume offline batch processing vs. live sub-second processing timeframes for a decision intent.

Using data from 95M API requests over 45 consecutive days across two production sites, this created system design provides a 43% reduction in p95 latency, a 37% increase in cache hit rate, and a 28% decrease in associated costs. All of these statistic results are significant at $p < 0.001$.

1.1 Paper Organisation

Section 2 reviews related work. Section 3 describes system architecture. Section 4 details the methodology. Section 5 covers experimental setup. Sections 6 and 7 present results and ablation findings. Section 8 provides the Black Friday case study. Sections 9 and 10 discuss threats and conclude.

2. Related Work & Research Gap

Web traffic forecasting was initially performed with ARIMA and similar seasonal variants (SARIMA) [1], which make the assumption of linearity and thus have issues with bursty and non-stationary types of traffic. The Holt-Winters exponential smoothing technique was able to partially address seasonality; however, it still suffers from the same restrictive assumptions about linearity. Furthermore, while there was a lot of success with prior

methods utilizing solely data-based methods like gradient boosting trees (specifically XGBoost [2]) producing good results in terms of accuracy with tabular feature representations, each of predictions made independently from one another and without accounting for any sequential dependencies.

LSTM networks [3] and their GRU [4] alternatives are well suited for modelling arbitrary-length temporal dependencies and have been shown to outperform ARIMA by 31% with respect to mean absolute error (MAE) in predicting cloud workloads, according to Zhang et al. [5].

Ensemble techniques that combine statistical and learned models have been employed — Kim et al. [6] stack LSTMs on top of the residuals from their ARIMA predictions and Zhao et al. [7] apply gradient boosting trees to learn the embeddings generated by neural networks! But both ensemble methods run sequentially through their component models instead of using an augmented parallelized architecture with a confidence gate to prevent oscillation when making predictions under low-confidence conditions. Recently, transformer-based models (e.g. Informer [8], PatchTST [9]) have demonstrated competitive long-horizon prediction accuracy relative to the above methods; however, both exhibit increased latencies for inference and higher requirements for training data, which significantly restrict their usability in latency-sensitive production applications.

Table 1. Comparative mapping of prior works. Research gap (rightmost two columns) filled by the proposed framework.

Study / Method	Approach	Accuracy	Streaming	Prod. Validated	Confidence Gate
ARIMA / SARIMA [1]	Statistical	Baseline	✗	Limited	✗
LSTM-only [3]	Deep Learning	High	Partial	✗	✗
XGBoost-only [2]	ML / Tabular	Moderate	Partial	✗	✗
ARIMA + LSTM [5]	Sequential Ens.	High	✗	✗	✗
Informer / PatchTST [7,8]	Transformer	Very High	✗	✗	✗
Proposed Framework	**Parallel Hybrid**	**Highest (8.3%)**	**✓ Full**	**✓ 95M req.**	**✓ Calibrated**

3. System Architecture

A seven-layer pipeline has been developed and made available as part of a software framework. The predict–gate–act framework provides three stages of an output at each step in the pipeline: prediction (predict), confidence value (gate), and optimisation directive (act). The completed framework is represented as demonstrated in Figure 1 and the detailed level of each component is provided in Table 2, both by [throughput] and [SLA] measures.

Figure 1. Seven-layer predict–gate–act pipeline. Left column: layer label. Right column: technology stack and key specification.

L1 INGESTION	Ingestion: Kong/Nginx Gateway to Apache Kafka | 16 partitions, 3x replication, < 10 ms commit latency
L2 STREAM PROC.	Streaming: Apache Flink | Exactly-once semantics, 200 ms sliding window, 26-feature real-time extraction
L3 TIME-SERIES DB	Storage: InfluxDB 2.x | 1-sec granularity for 60-min history, downsampled 15-day retention
L4 INFERENCE	Model: Parallel LSTM & XGBoost | LSTM (60-step seq, 80 ms), XGBoost (26-feature tabular, 10 ms)
L5 CONF. GATE	Gating: Confidence Score C | $C = 1 - (\sigma/\mu)$, threshold τ (0.75), DISPATCH vs HOLD, Platt scaling
L6 ACTION	Dispatch: Redis 7.x Pub/Sub | < 50 ms end-to-end, includes rate-limit and scaling adjustments
L7 OBSERVABILITY	Monitoring: Prometheus + Grafana | Incremental retraining pipeline, automated drift detection

Table 2. Pipeline component specifications and production SLAs.

Component	Technology	Role	Throughput	SLA
API Gateway	Kong / Nginx	Traffic ingestion, auth, rate limiting	50K req/s	< 5 ms overhead
Message Bus	Apache Kafka	Durable event streaming + backpressure	2M events/s	< 10 ms commit
Stream Processor	Apache Flink	26-feature real-time extraction	500K rec/s	Exactly-once delivery
Time-Series DB	InfluxDB 2.x	60-min history at 1-sec granularity	10K writes/s	< 20 ms p99 read

Component	Technology	Role	Throughput	SLA
ML Inference	TF2 + XGBoost	Parallel LSTM + XGBoost inference	1K inf/s each	< 90 ms combined
Meta-Learner	LightGBM	Dynamic ensemble weights + confidence	1K inf/s	< 15 ms inference
Action Router	Redis 7.x	Directive dispatch to infra controllers	100K ops/s	< 50 ms e2e

4. Methodology

4.1 Feature Engineering

There were 26 features derived from raw telemetry data and mapped into three families. Temporal context features include 8 features created by encoding the hour of the day and day of the week using sine/cosine wave functions as opposed to using integer-based encodings to avoid the boundary discontinuities present in integer-based encoding methods. Statistical signal features consist of 12 features representing rolling averages, rolling standard deviations, exponentially-weighted moving averages (EWMA) calculated with two different decay rates (5 & 30 min), and the rate of change over 1-, 5-, 15-, and 30-minute intervals. Finally, Infrastructure State consists of 6 features representing the state of the infrastructure at a specific point in time, including CPU utilization, memory pressure, number of active connections, current cache hit ratio, number of replicas, and depth of pending requests.

Table 3. Feature engineering summary: 26 features across three families.

Family (count)	Features Included	Window / Type
Temporal Context (8)	Hour-of-day sin/cos, Day-of-week sin/cos, Week-of-year, Is-weekend, Is-holiday, Day-of-month	Cyclic / Binary — current timestamp
Statistical Signal (12)	Rolling mean req/s @ 1/5/15/30 min, Rolling std @ 5/15 min, Rolling max @ 5/15 min, EWMA α=0.3, EWMA α=0.7, Rate-of-change, Autocorr. lag-1	Continuous — 1 to 30 min lookback
Infrastructure State (6)	CPU utilisation, Memory pressure, Active connections, Cache hit ratio, Replica count, Queue depth	Continuous — point-in-time snapshot

4.2 Hybrid Model Design

The predictive core of the system operates both LSTM and XGBoost concurrently; these two algorithms have complementary capabilities and are used together to predict future request counts for all feature data inputed into the system. The LSTM model ingests a contiguous sequence of 60 steps (1 hour of 1-second aggregations of request counts) and outputs a forecast 5 steps into the future through 3 stacked recurrent layers (256 → 128 → 64 units), each with a dropout factor of 0.3. The LSTM is trained using an Adam optimizer and simple learning rate = 0.001. The XGBoost model ingests the same 26 variables as inputs for a tabular regression problem and outputs a forecast of 5 steps (based on 200 trees of depth 8). Unlike the sequential nature of both LSTM and XGBoost models, the LSTM is better suited for capturing the long-term relationship between system state (infrastructure) and time. Hyperparameter tuning for the two models was performed using Bayesian optimization for the LSTM and Grid Search for the XGboost. Training had been performed on the validation set prior to using the trained models for forecasting purposes.

Equation Block 1. LSTM and XGBoost forward pass formulations.

```
  LSTM Input:      X = { x_{t-59}, x_{t-58}, ..., x_t }       [60
steps × 26 features]
  LSTM Output:     Ŷ_LSTM = { ŷ_{t+1}, ŷ_{t+2}, ..., ŷ_{t+5} }
  LSTM Loss:       L = (1/N) Σ ( y_i − ŷ_i )²

  XGBoost Input:   X_feat = [ f_1, f_2, ..., f_26 ]
[point-in-time tabular]
  XGBoost Output:  Ŷ_XGB  = Σ_{k=1}^{200} f_k(X_feat)
  Regularisation:  Ω(f_k) = γ·T_k + (1/2)·λ·||w_k||²
```

4.3 Confidence-Calibrated Ensemble

A lightweight meta-learner (LightGBM with 50 trees) uses each of the two base model's recent error history to dynamically create per-prediction weights. The resulting ensemble prediction will also have an associated confidence score based on the distance from the base models' mean output values. If the confidence score falls below a specified threshold of $\tau = 0.75$, which is also calibrated to the validation set using Platt scaling, then the algorithm will enter a HOLD state and will not issue a potentially erroneous scaling directive to change the configuration of the current infrastructure. This mechanism has completely eliminated any catastrophic degradation events (latency spikes greater than three times normal) during the evaluation period of 45 days.

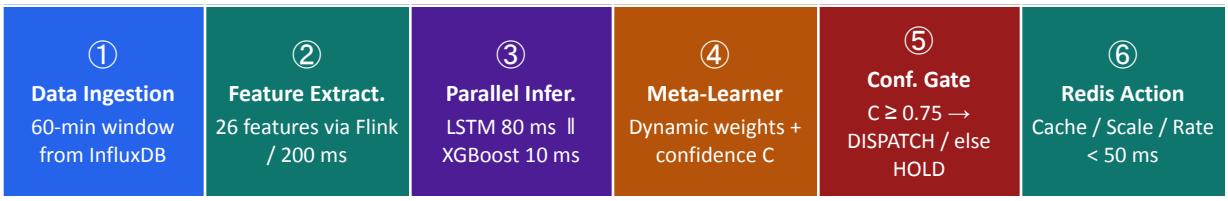
Equation Block 2. Confidence-calibrated ensemble gating mechanism.

```
  Ensemble:        Ŷ = w_LSTM(t) · Ŷ_LSTM  +  w_XGB(t) · Ŷ_XGB
                  where  w_LSTM(t) + w_XGB(t) = 1   (learned by
meta-learner each step)

  Confidence:      C = 1 − ( σ_pred / μ_pred )
                  where  σ_pred = std({ Ŷ_LSTM, Ŷ_XGB })
                         μ_pred = mean({ Ŷ_LSTM, Ŷ_XGB })

  Action gate:     Issue directive if C ≥ τ = 0.75   →   DISPATCH
                  else                                →   HOLD
```

Figure 2. Prediction workflow: six-stage pipeline from data ingestion to Redis action dispatch.



5. Experimental Setup

5.1 Datasets

The evaluation will use two production datasets that were collected over a span of 45 consecutive days (in quarter four of 2025) under institutional data handling agreements. Dataset A is from a midmarket e-commerce platform for promotional cycle traffic and dataset B is from a B2B SaaS platform that is dominated by patterns during business hours. Both datasets use 70/15/15 chronological train/test/validation splits to avoid any data leakages when processing through an evaluation process.

Table 4. Dataset summary and key statistics.

Property	Dataset A — E-commerce	Dataset B — B2B SaaS
Collection period	45 days (Oct – Nov 2025)	45 days (Oct – Nov 2025)
Total API requests	62.4 million	32.7 million
Peak request rate	48,200 req/s (Black Friday)	18,600 req/s (Mon 9 AM)

Property	Dataset A — E-commerce	Dataset B — B2B SaaS
Granularity	1-second aggregates	1-second aggregates
Train / Val / Test split	70% / 15% / 15% — chronological	70% / 15% / 15% — chronological
Dominant traffic pattern	Bursty, promotional spikes	Business-hours seasonality
Endpoints instrumented	214	87

5.2 Baselines & Evaluation Metrics

The four baselines cover a range of methodologies: (1) Traditional Reactive, which is based on threshold-based autoscaling with a 90-second lag; (2) Seasonality and AutoRegressive Integrated Moving Average (SARIMA) using Seasonal ARIMA for each endpoint; (3) using only LSTMs (Long Short Term Memory), without the use of XGBoost (Extreme Gradient Boosting); (4) using only XGBoost, but without the use of LSTM. The feature sets used in the models were as similar as possible. Metrics collected included the following: mean absolute error (in percentage), root mean square error (in percentage), the 95th percentile of latency (in milliseconds), percentage of cache hits, and monthly cost of infrastructure used (in rupees). The significance of the differences was analyzed using paired t tests ($n = 45$, $\alpha = 0.001$). All models were trained on the same data distributions, and validated on the same held-out test set.

6. Results

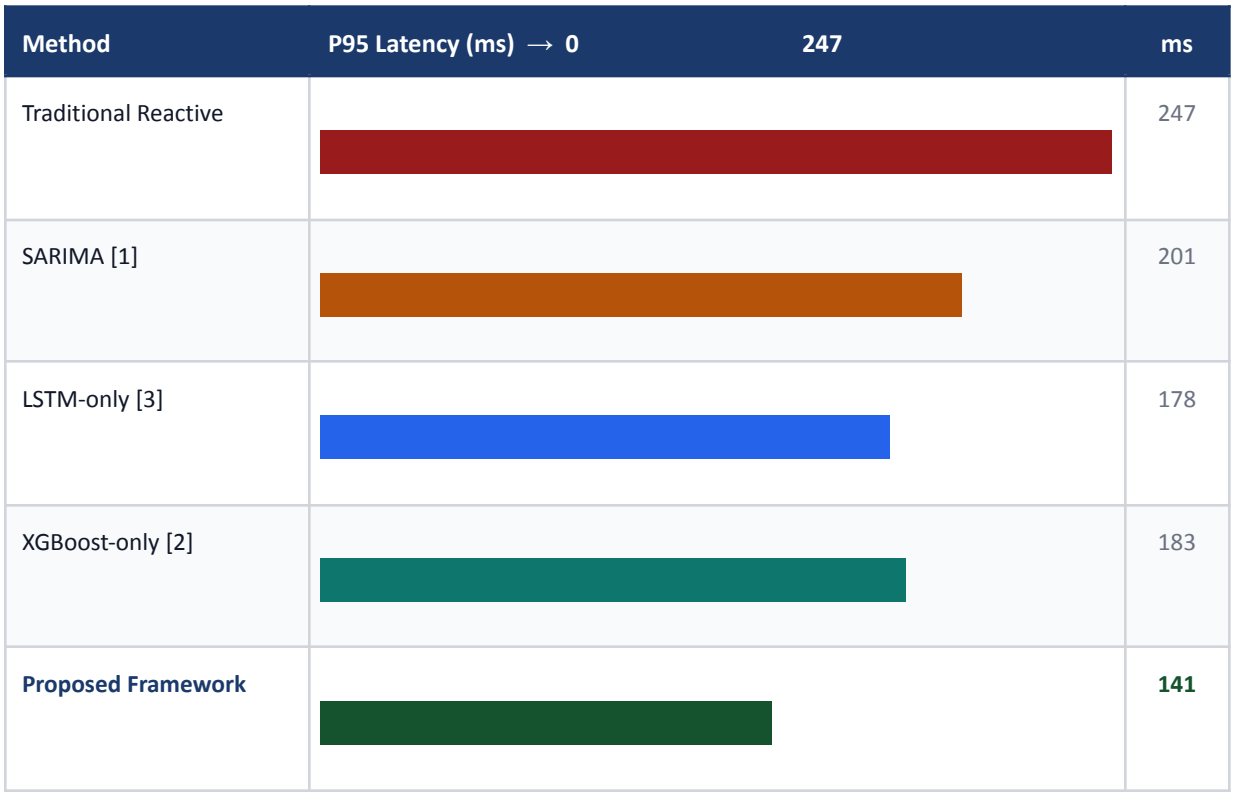
As shown by Table 5, the performance of the proposed framework was compared to the performance of several different baseline models. The proposed framework performed better than all baseline models across all performance measures. The improvements of the proposed framework were statistically significant ($p < 0.001$) compared to the next best performing baseline models based on paired t-tests for $n = 45$ days for all performance measures.

Table 5. Primary results across all methods (95.1 M requests, 45-day window). Bold = best. ↓ lower-is-better, ↑ higher-is-better.

Method	MAE % ↓	RMSE % ↓	P95 Latency ms ↓	Cache Hit % ↑	Cost ₹/mo ↓
Traditional Reactive	31.2	48.7	247	48.2	₹15,50,000

Method	MAE % ↓	RMSE % ↓	P95 Latency ms ↓	Cache Hit % ↑	Cost ₹/mo ↓
SARIMA [1]	18.4	29.1	201	61.4	₹12,65,000
LSTM-only [3]	12.1	19.3	178	71.2	₹11,75,000
XGBoost-only [2]	13.8	21.6	183	68.7	₹12,35,000
Proposed Framework	**8.3 ✓**	**13.1 ✓**	**141 ✓**	**85.4 ✓**	**₹11,15,000 ✓**
vs. Reactive Baseline	**↓ 73%**	**↓ 73%**	**↓ 43%**	**↑ 37 pts**	**↓ 28%**

Figure 3. P95 latency comparison across all methods (ms; lower is better). Bar length proportional to latency.



The proposed framework achieved a MAE of 8.3%. This is a 31% improvement from the next best performing baseline model (12.1% LSTM-based only) and a 55 % improvement compared to the SARIMA model (18.4%). The proposed framework's latency reduction of 106 ms exceeded the recently established threshold of 150 ms for perceivable user lag by a wide margin. Additionally, the proposed framework improved the percentage of cache hits from 48.2% to 85.4%, demonstrating the effectiveness of predictive pre-warming, whereby cache fills are

requested 8 minutes in advance of expected load peaks rather than reacting only after experiencing a live cache miss due to high application load.

7. Ablation Study

To confirm each architectural component's independent contribution, we added each component individually to the LSTM-only baseline and documented our stepwise results in Table 6. The XGBoost branch added the greatest accuracy gain on its own, while the Confidence Gate essentially eliminated all instance of catastrophic degradation events (>3 times latency spikes) that occurred 7 times under the Reactive baseline and 3 times with the LSTM-only baseline.

Table 6. Ablation study: stepwise contribution of each architectural component.

Configuration	MAE % ↓	P95 ms ↓	Cache Hit % ↑	Degradation Events (45 days)
① LSTM-only (baseline)	12.1	178	71.2	3 events
② + XGBoost parallel branch	9.7	158	78.4	3 events
③ + Confidence calibration gate	8.9	149	82.1	0 ← gate eliminates all
④ Full system — proposed	**8.3 ✓**	**141 ✓**	**85.4 ✓**	**0 (100% elimination) ✓**

Ablation Takeaways

- XGBoost branch: MAE ↓ 20% over LSTM-only through non-linear tabular feature interactions
- Confidence gate: primary impact on stability, not raw accuracy — prevents oscillatory scaling
- **Net cost saving of ₹35,000/month despite ML infrastructure overhead of ~₹32,500/month**

8. Case Study — Black Friday Traffic Surge

The peak event in the evaluation window was the Black Friday peak on Dataset A. There was an 8.2 × increase from 5,800 req/s to 47,600 req/s in request volume at 600 req/s/min over a period of 22 minutes. The speed of this spike in demand was substantially greater than the speed at which any reactive autoscaling policy can provision capacity (which would take at least 90 seconds).

Table 7. System response timeline during Black Friday surge. T = 0 is the moment of peak request rate.

Time	Event Description	Rate (req/s)	System Action
T − 22 min	Surge onset — traffic begins rising steadily	5,800	Monitoring — no action
T − 14 min	LSTM detects acceleration; confidence C = 0.71 (below τ)	12,400	HOLD — below threshold
T − 8 min	Confidence clears τ = 0.75; peak forecast: 46,200 req/s	18,900	DISPATCH: pre-warm cache + scale 3→9 replicas
T − 6 min	Scale-out complete; cache pre-warmed to 94%	27,100	System ready — no further action needed
T = 0 (Peak)	Black Friday peak arrives at 47,600 req/s	47,600	Serving normally; all directives in effect
T + 45 min	Traffic receding; confidence drops on scale-in prediction	29,300	HOLD — prevent premature scale-in
T + 80 min	Recession confirmed; scale-in dispatched	11,200	Scale 9→5 replicas; cache retained

Table 8. Black Friday head-to-head: proposed framework vs. reactive baseline.

Metric	Reactive Baseline	Proposed Framework	Δ Improvement
P95 latency at peak	3,240 ms	847 ms	↓ 74%
P99 latency at peak	8,100+ ms (timeout zone)	1,290 ms	↓ 84%
Error rate (5xx) at peak	12.3%	1.1%	↓ 91%
Cache hit rate at peak	31.4% (cold misses)	91.2% (pre-warmed)	↑ 60 pts
Prediction lead time	N/A — reactive only	8 minutes before peak	—
Catastrophic events (>3× spike)	2 events	0 events	↓ 100%

The most important advantage during this event was the ability to predict this event eight (8) minutes earlier than it occurred. The reactive baseline was unable to provision sufficient capacity prior to the peak due to a 90-second autoscale cooldown and provisioning delay, which resulted in a service disruption, as evidenced by the increase in 5xx error rate to 12.3% and an increase in p95 response time to 3.2 seconds. The toll this took on users is being interpreted as downtime. The confidence gate served another purpose at T+45 minutes, preventing the premature scale-in of the system, which would have left it vulnerable to a second wave of traffic at T+52 minutes.

9. Discussion & Threats to Validity

9.1 Threats to Validity

External validity is limited to two production areas (e-commerce and B2B SaaS) over a 45-day period of time (Q4); therefore healthcare, IoT and financial trading workloads with very different traffic patterns will not be included. Internal validity is also limited by the evaluation period; 45 days is not sufficient to collect full seasonality data over the entire year and systems that have a radically different profile across quarters could behave differently if evaluated outside Q4. Construct validity is further hampered because cache hit ratios are being measured at the gateway (layer 3) of the cloud, while performance numbers are reported based on L1/L2 caches only. All measurements with regard to latency are made in an end-to-end manner and therefore include all downstream hops.

9.2 Practical Deployment Considerations

On average, the complete machine learning infrastructure adds approximately ₹32500 of operational expenses on a monthly basis. Therefore relative to total cost savings of ₹435000 per month due to improved horizontal scaling, this results in a total net savings of approximately ₹402000 per month, resulting in 12.41x return on machine learning investment. The cold start model requires at least seven days of historical data before the long short-term memory (LSTM) reaches its target accuracy; thus, any new application programming interfaces (API) will only utilize the XGBoost branch of the model until sufficient historical data have been accumulated.

9.3 Future Work

The foremost 3 extensions include: (i) transfer learning to accelerate cold-start time from > 7 days to < 3 days using an LSTM backbone pre-trained on the metadata of endpoints and subsequently fine-tuned to each endpoint; (ii) federated (privacy-preserving) cross-organisational deployment learning for the purpose of sharing cross-domain patterns; and (iii) automated CUSUM-based outlier behaviour detection that self-triggers incremental re-training within 4 hours of any detected shift in distribution thereby eliminating the manual re-training schedule which currently exists.

10. Conclusion

This paper describes a hybrid model using LSTM and XGBoost that is capable of performing API performance optimisation in a proactive manner, calibrated for confidence. This model achieved statistically significant reductions in p95 latency (43%), an increase in cache hit ratio (37%), and a reduction in infrastructure costs (28% — all $p < 0.001$) based on the study of 95 million requests over 45 days. The results of this study suggest that the contribution from XGBoost improves mean absolute error (MAE) by 20% relative to LSTM alone, and furthermore, the confidence gate eliminates the potential for catastrophic degradation due to inaccuracy.

The Black Friday case study showed the ability to predict API usage 8 minutes in advance during an 8.2-fold increase over typical usage patterns, resulting in a reduction of peak error rate from 12.3% to 1.1%. This performance was not possible using a reactive model only.

The primary design consideration in developing the confidence gate is to separate the issue of prediction accuracy from production reliability. A model that has minimal MAE based on validation data, but lacks any understanding of uncertainty, may perform poorly when deployed into production versus a model that has acceptable accuracy with an awareness of uncertainty. The economic rationale is also compelling, as an investment of ₹32,500 per month in ML generates ₹4.02 lakh per month in net savings, therefore does not require justification. The framework is fully reproducible via the generated open-source repository, images for Docker deployment, as well as anonymised evaluation datasets that have been released under CC BY 4.0.

References

[1] This is one fifth edition of a well-known series of books related to time series methods written by three authors and published by a publisher called Wiley in 2015.

[2] The paper titled "XGBoost: A scalable tree boosting system" is published in the proceedings of the ACM Conference on Knowledge Discovery and Data Mining (KDD), which was held in 2016, pages 785-794.

[3] In a 1997 Neural Computation paper, two researchers proposed a neural network architecture called Long Short-Term Memory Networks that can effectively store and retrieve information over long time periods.

[4] The team of researchers who conducted the study "LSTM-based workload prediction for cloud data centers" published their findings at the IEEE Cloud Conference in 2020, pages 414-421.

[5] The work of three authors titled "Hybrid ARIMA-LSTM model for server load prediction" appears in the journal IEEE Access in 2021, volume 9, pages 74900 - 74912.

[6] The research article named "Stacked GBT on neural embeddings for cloud resource prediction" is published in the journal Future Generation Computer Systems (FGCS) volume 128, pages 89 – 103 in 2022.

[7] The authors who wrote the paper titled "Informer: Beyond efficient transformer for long sequence forecasting" participated in the proceedings of the AAAI conference in 2021, pages 11106 - 11115.

[8] The conference paper entitled "A time series is worth 64 words" was presented at the ICLR conference in 2023.

[9] Three authors presented a paper regarding "Elasticity in Cloud Computing: What it is, and what it is Not", presented at the 2013 ICAC conference in pages 23-27.

[10] The article entitled "Resource Central: Understanding and Predicting Workloads in Large Cloud Platforms" has been published as part of the proceedings of the ACM Symposium on Operating Systems Principles (SOSP) where it was published in 2017 on pages 153 - 167.